Mini Vision Transformer Interview Modification Guide

For image classification datasets in ImageFolder format: Data/train/class_x and Data/val/class_x

1. What this template is for

This MiniViT template is designed for image classification tasks such as ants/bees, cat/dog, or normal/defect classification. It uses torchvision.datasets.ImageFolder to infer class names from folder names, then trains a lightweight Vision Transformer with validation, checkpoint saving, training-curve plotting, and single-image inference.

```
Data/
■■■ train/
■   ■■■ cat/
■   ■■■ dog/
■■■ val/
    ■■■ cat/
    ■■■ dog/
```

If the dataset follows this structure, most interview modifications only happen in the configuration section.

2. First inspect the dataset structure

After receiving a dataset, first check the root folder, train/val folders, and class folders.

```
DATA_DIR = "Data"
print(os.listdir(DATA_DIR))
print(os.listdir(os.path.join(DATA_DIR, "train")))
print(os.listdir(os.path.join(DATA_DIR, "val")))
```

If the output looks like [train, val], [cat, dog], [cat, dog], your template can read the dataset directly.

3. Configuration items you must modify flexibly

Parameter	How to modify it during the interview
DATA_DIR	Change it to the provided dataset path, for example "Data".
OUTPUT_DIR	Change it to your output folder, for example "outputs_mini_vit".
IMG_SIZE	Use 64 for CPU; 128 is possible on GPU. Larger images create more patches and slower training.
PATCH_SIZE	Use 8 for CPU or 16 for GPU. IMG_SIZE must be divisible by PATCH_SIZE.
IN_CHANNELS	Use 3 for RGB images. Use 1 for grayscale images and add Grayscale transform.
D_MODEL	Use 64 for CPU or 128 for GPU. It must be divisible by NHEAD.
NHEAD	Usually 4 is enough. D_MODEL / NHEAD must be an integer.
NUM_LAYERS	Use 1 for CPU or 2 for GPU. More layers are slower.
BATCH_SIZE	Use 2 or 4 for CPU; 8 or 16 for GPU.
NUM_EPOCHS	Use 1-3 for interview demos; 5 if GPU is available.
NUM_WORKERS	Use 0 for Windows/Jupyter safety; try 2 on Linux if stable.

4. Recommended settings

CPU-friendly interview setting:

```
DATA_DIR = "Data"
IMG_SIZE = 64
PATCH_SIZE = 8
IN_CHANNELS = 3
D_MODEL = 64
NHEAD = 4
NUM_LAYERS = 1
DIM_FEEDFORWARD = 128
BATCH_SIZE = 4
NUM_EPOCHS = 3
NUM_WORKERS = 0
LEARNING_RATE = 1e-3
```

GPU-friendly interview setting:

```
IMG_SIZE = 128
PATCH_SIZE = 16
D_MODEL = 128
NHEAD = 4
NUM_LAYERS = 2
DIM_FEEDFORWARD = 256
BATCH_SIZE = 8
NUM_EPOCHS = 5
```

5. Why IMG_SIZE and PATCH_SIZE matter

For Vision Transformers, the input sequence length is determined by the number of patches. More patches mean much slower self-attention.

```
num_patches = (IMG_SIZE / PATCH_SIZE) * (IMG_SIZE / PATCH_SIZE)

# Example 1: IMG_SIZE=64, PATCH_SIZE=8
# num_patches = 8 * 8 = 64
# sequence length = 64 + 1 CLS token = 65

# Example 2: IMG_SIZE=224, PATCH_SIZE=8
# num_patches = 28 * 28 = 784
# sequence length = 785, much slower on CPU
```

Do not start with 224x224 images and patch size 8 in a CPU-based interview demo.

6. How data loading works

The build_dataloaders() function uses ImageFolder to automatically read class names. You do not need to manually set NUM_CLASSES.

```
image_datasets = {
    phase: datasets.ImageFolder(
        root=os.path.join(data_dir, phase),
        transform=data_transforms[phase]
    )
    for phase in ["train", "val"]
}

class_names = image_datasets["train"].classes
num_classes = len(class_names)
```

If Data/train contains cat, dog, and rabbit folders, the model output dimension automatically becomes 3.

7. The model body usually does not need changes

PatchEmbedding uses Conv2d to perform patch splitting and feature projection. MiniViT uses a class token, positional embedding, TransformerEncoder, and final classification head. During interviews, control the model size through configuration values rather than rewriting the model.

```
Image
  -> PatchEmbedding
  -> CLS token + Positional Embedding
  -> Transformer Encoder
  -> CLS token output
  -> Linear classification head
  -> class logits
```

8. The training code usually does not need changes

For classification, use CrossEntropyLoss. The model forward pass should return raw logits. Do not manually apply softmax inside the model forward function because CrossEntropyLoss handles it internally.

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY)

outputs = model(inputs)
loss = criterion(outputs, labels)
_, preds = torch.max(outputs, dim=1)
```

During validation, use torch.no_grad() to avoid gradient computation and reduce memory usage.

9. What if there is no train/val split?

If the dataset looks like Data/cat and Data/dog instead of Data/train/cat and Data/val/cat, you need to split it first or add an automatic split function. In an interview, the clean answer is: inspect the dataset first; if no train/val split exists, use random_split.

```
from torch.utils.data import random_split

full_dataset = datasets.ImageFolder(root=DATA_DIR, transform=data_transforms["train"])
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size])
```

10. What if the images are grayscale?

Keep IN_CHANNELS = 3 for standard RGB images. For grayscale or medical images, set IN_CHANNELS = 1 and add a Grayscale transform.

```
IN_CHANNELS = 1

transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.Grayscale(num_output_channels=1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])
```

11. How to modify single-image inference

The template automatically picks one validation image. If needed, you can manually specify the test image path.

```
# Automatically pick one validation image
test_img_path = os.path.join(DATA_DIR, "val", class_names[0],
                             os.listdir(os.path.join(DATA_DIR, "val", class_names[0]))[0])
```

```
# Manually specify one image
test_img_path = "Data/val/cat/cat_001.jpg"
```

12. Interview plan you can say directly

I will solve this as an image classification task using a lightweight Vision Transformer. First, I will inspect the dataset structure and check whether it follows the ImageFolder format with train and validation folders. Then I will resize all images, apply simple augmentation for training, and normalize them. Class names and number of classes will be inferred automatically from folder names.

For the model, I use a convolution-based patch embedding layer, then add a learnable class token and positional embedding. The token sequence is processed by a Transformer encoder, and the class token output is used for classification. Since this is an interview setting and may run on CPU, I use a small image size, small embedding dimension, and one Transformer encoder layer. Finally, I train with CrossEntropyLoss and AdamW, evaluate validation accuracy, save the best checkpoint, and visualize predictions.

13. Common follow-up questions

Why use patch embedding?

A Transformer expects sequence input, while an image is a 2D grid. Patch embedding splits the image into patches and maps each patch into a token vector.

Why use a class token?

The class token is a learnable token added to the patch sequence. After the Transformer encoder, its output can represent the whole image for classification.

Why use positional embedding?

Self-attention itself does not know the spatial order of patches. Positional embedding provides location information.

Why use AdamW?

AdamW is commonly used for Transformers because it provides adaptive learning rates and decoupled weight decay, often improving optimization stability.

Why not use ResNet?

ResNet transfer learning may be stronger for small datasets, but this template demonstrates a Transformer-based vision pipeline. In practice, I would compare both ResNet and MiniViT.

14. Final interview modification checklist

- Set DATA_DIR to "Data" or the actual dataset path.
- Confirm ImageFolder format: train/val/class_name.
- For CPU: IMG_SIZE=64, PATCH_SIZE=8, D_MODEL=64, NUM_LAYERS=1.
- For GPU: IMG_SIZE=128, PATCH_SIZE=16, D_MODEL=128, NUM_LAYERS=2.
- Use NUM_WORKERS=0 on Windows/Jupyter.
- Use IN_CHANNELS=3 for RGB images and IN_CHANNELS=1 for grayscale images.

• Do not manually write the number of classes; use len(class_names).

• Modify test_img_path only if you want a specific test image.